

Enterprise Fraud Detection: Detecting Financial Fraud in Real Time

Israel De La Mothe*, Kieron Seepersad*, Dylan Sinkia*, and Tyrese Des Vignes*

*Department of Computing and Information Technology

The University of the West Indies, St. Augustine Campus

Email: {israel.delamothe, kieron.seepersad, dylan.sinkia, tyrese.desvignes}@my.uwi.edu

Abstract—Financial institutions process millions of transactions daily, making real-time fraud detection increasingly challenging as fraudsters evolve tactics including coordinated rings and account takeovers. Traditional rule-based systems struggle to detect emerging patterns, and extreme class imbalance (often less than 0.2% fraudulent transactions) which renders standard classification models ineffective.

An end-to-end big data pipeline was built that ingests the PaySim synthetic mobile money dataset (6.36 million transactions, 0.13% fraud rate), engineers 34 behavioral, temporal, and balance-based features, and trains a gradient-boosted classifier to identify suspicious transactions. The best model, LightGBM with class weighting (scale_pos_weight = 773), achieves a PR-AUC of 0.9986 and an F1 score of 0.9988 on a held-out test set. We deploy the model behind a FastAPI service and a Streamlit dashboard that surfaces predictions. This paper reports the methodology, results, and engineering trade-offs.

Index Terms—big data, fraud detection, LightGBM, gradient boosting, MLOps, imbalanced learning

I. INTRODUCTION AND PROBLEM STATEMENT

Financial fraud remains one of the most persistent and costly challenges facing modern financial institutions. Globally, on-line payment fraud alone is projected to exceed \$98 billion per year by 2028, with losses growing annually as digital transactions proliferate. Traditional rule-based fraud detection systems suffer from two fundamental limitations: they cannot adapt to emerging fraud patterns, and they generate an unmanageable volume of false positives that overwhelm investigative teams.

Fraudsters have evolved beyond simple single-account attacks. Contemporary fraud schemes include coordinated fraud rings, where multiple accounts execute synchronized small-value transactions to avoid detection, and account takeover attacks, where legitimate customer credentials are compromised and used for unauthorized transfers. These patterns are inherently difficult to capture with static rules because they exploit the very characteristics that define normal customer behavior: small amounts, familiar locations, and routine transaction frequencies.

The extreme class imbalance presents the first major technical challenge. A classifier that predicts "legitimate" for every transaction achieves [99.87%] accuracy while detecting exactly zero fraud. Standard machine learning models optimized for accuracy fail catastrophically on such data.

The second challenge is scale. Financial institutions process millions of transactions daily. Any fraud detection system must

score transactions with sub-second latency to be useful operationally. This precludes computationally expensive models and requires efficient feature engineering and inference pipelines.

The third challenge is explainability. Fraud investigators need to understand why a transaction was flagged. A black-box model that produces a score without justification is unusable in practice because investigators cannot verify its reasoning, and regulators require auditable decisions.

The PaySim dataset contains 6.36 million transactions, which is too large for single-machine memory, requiring efficient columnar processing with Polars and Parquet. The 0.13% fraud rate renders naive machine learning approaches useless, demonstrating why accuracy is misleading for imbalanced problems. Finally, the techniques developed here, class weighting, threshold tuning, PR-AUC evaluation, and SHAP explainability, generalize beyond fraud detection to any imbalanced binary classification task.

II. DATA DESCRIPTION AND SOURCE

A. Primary dataset

We use the PaySim synthetic mobile money dataset, developed by Edgar Lopez-Rojas [1], and publicly available through Kaggle. PaySim is a simulation of mobile money transactions based on real transaction data from a large African mobile money service. The simulator was calibrated using anonymised financial logs to replicate the statistical properties of legitimate and fraudulent transactions while preserving privacy. The dataset contains 6,362,620 transactions which spans 30 simulated days. Each transaction record includes 11 columns: Transaction ID, step (hour of simulation), type, origin name, old and new balances from origin, destination name, old and new balances for destination, is fraud, and is flagged fraud. The target variable isFraud has a heavily imbalanced distribution, 0.13% fraudulent, making this imbalance the central technical challenge.

B. Data quality and cleaning

PaySim is a synthetic dataset with no missing values. However, we performed several validation and preprocessing steps to ensure data integrity and prepare the data for modeling.

Duplicate detection. We verified that all transaction identifiers (implicitly, the combination of nameOrig, nameDest, step, and amount) are unique. No duplicate records were found.

Constraint validation. We checked for impossible values: negative amounts, negative balances, and transactions where the sender had insufficient funds (`oldbalanceOrg > amount`). No negative amounts or negative balances were present. Insufficient balance transactions (where `oldbalanceOrg > amount`) occur in the dataset but are retained because they represent legitimate overdraft or failed transaction scenarios.

Exclusion of Leakage-Prone Features. The dataset includes an `isFlaggedFraud` column, which is a rule-based flag indicating whether a transaction exceeded a predefined amount threshold. This column was excluded from the feature set prior to modeling. Including it would constitute data leakage, as it is derived directly from the target variable’s underlying logic. A model trained on this column would simply memorize the rule and fail to generalize to any fraud pattern not captured by that static threshold.

Skewness Assessment. We examined the distribution of the amount and balance columns (`oldbalanceOrg`, `newbalanceOrig`, etc.). These distributions were highly right-skewed, with most values concentrated near zero and a long tail of large transactions. This assessment motivated the application of log transforms during feature engineering. No rows were dropped during the cleaning process.

III. LITERATURE REVIEW

Fraud detection in financial transactions has been studied extensively across three methodological families: rule-based systems, anomaly detection, and supervised machine learning. Each approach has distinct strengths and limitations that inform our work.

Traditional fraud detection relies on static rules: flagging transactions that exceed monetary thresholds, originate from high-risk jurisdictions, or occur outside of a customer’s typical geographic region [2]. Financial institutions increasingly supplement rules with machine learning models.

Anomaly detection methods identify transactions that deviate from learned normal behavior. Bolton and Hand [3] and proposed peer group analysis, where a customer’s transaction is compared to similar customers. Liu et al. [4] demonstrated that isolation forests are effective for high-dimensional anomaly detection. However, these methods cannot distinguish rare legitimate behavior from actual fraud.

Supervised machine learning substantially outperforms both approaches when labeled data is available. Bhattacharyya et al. [5] compared multiple classifiers on credit card data, finding that random forests achieve the best precision-recall balance. For extreme class imbalance (fraud rates $\leq 0.5\%$), cost-sensitive learning, weighting positive class errors by the imbalance ratio is more effective than sampling methods like SMOTE [6].

Gradient boosting has become the industry standard for fraud detection. LightGBM [7] and XGBoost [8] handle imbalanced data efficiently via `scale_pos_weight` and `scale` to millions of transactions. LightGBM’s leaf-wise tree growth offers faster training on large datasets.

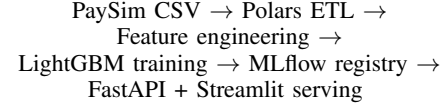


Fig. 1. End-to-end pipeline from raw PaySim data to deployed fraud scoring API.

Evaluation metrics. Accuracy is misleading for imbalanced problems; a classifier that predicts “legitimate” for all transactions achieves 99.87% accuracy while detecting zero fraud. Davis and Goadrich [9] established that the Precision-Recall curve and PR-AUC are more informative than ROC curves for imbalanced classification.

Explainability. Regulators increasingly require automated decisions to be explainable. Lundberg and Lee [10] introduced SHAP, a unified framework based on cooperative game theory that computes each feature’s contribution to a specific prediction. SHAP enables investigators to validate fraud flags.

Our work uses LightGBM with cost-sensitive learning, evaluates using PR-AUC and F1 score, and provides SHAP explanations for each prediction.

IV. METHODOLOGY

A. Pipeline overview

Figure 1 shows the end-to-end pipeline. Raw PaySim CSV files are ingested using Polars, cleaned and transformed, and aggregated into a feature-rich dataset. LightGBM models are trained from this dataset; predictions are served by a FastAPI application that returns fraud probabilities and SHAP explanations for any transaction.

B. Feature engineering

The model consumes 34 features per transaction row:

- **Balance engineering features:** `orig_balance_error` (expected minus actual sender balance), `dest_balance_error` (expected minus actual receiver balance), `orig_zeroed` (sender balance equals zero after transaction), `dest_zeroed` (receiver balance equals zero), `sufficient_balance` (sender had enough funds), `dest_was_empty` (receiver balance was zero before transaction), `amount_ratio_orig` (amount divided by sender balance plus one), `amount_ratio_dest` (amount divided by receiver balance plus one), `exact_drain` (amount equals sender’s entire balance).
- **Time features:** `hour_of_day` (0-23), `hour_of_week` (0-167, hours within a 7-day week), `day` (0-30, day number since simulation start), `is_night` (binary flag for hours 0-5 or 22-23), `step` (original hour of simulation).
- **Log transforms:** `log_amount`, `log_oldbal_orig`, `log_newbal_orig`, `log_oldbal_dest`, `log_newbal_dest` (log1p applied to amount and balance columns to normalize skewed distributions).
- **Transaction type features:** `type_enc` (encoded transaction type: PAYMENT=0, TRANSFER=1, CASH_OUT=2, DEBIT=3, CASH_IN=4), `is_risky_type`

(binary flag for TRANSFER and CASH_OUT, the only transaction types where fraud occurs in PaySim).

- **Account type features:** `orig_is_customer` (sender identifier begins with 'C'), `dest_is_customer` (receiver identifier begins with 'C'), `dest_is_merchant` (receiver identifier begins with 'M').
- **Raw retained features:** `amount` (original amount, retained alongside log transform), `oldbalanceOrg`, `newbalanceOrig`, `oldbalanceDest`, `newbalanceDest` (original balance columns, retained alongside log transforms).

C. Model and training

We used LightGBM with 1000 boosting rounds, a learning rate of 0.05, and max leaves of 127. The data was split randomly into training (60%), validation (20%), and test (20%) sets using stratified sampling to preserve the 0.13% fraud rate across all splits. Because PaySim is synthetic and lacks genuine timestamps, we used random stratified splitting rather than temporal splitting. In a real-world deployment, temporal validation would be required to prevent look-ahead bias.

D. Baselines and evaluation

We compare against (i) a majority class baseline that predicts "legitimate" for every transaction. This baseline achieves 99.87% accuracy but detects zero fraud, demonstrating why accuracy is misleading for imbalanced problems, and (ii) a class-weighted XGBoost. An alternative gradient boosting implementation with the same `scale_pos_weight` parameter (773), included for comparison against LightGBM.

We report three primary metrics, computed over all transactions in the held-out test set: PR-AUC (Precision-Recall Area Under Curve), F1 score, and Recall (Sensitivity).

We additionally report the confusion matrix (true positives, false positives, true negatives, false negatives) and classification report for transparency.

E. Serving and reproducibility

Trained models are registered in MLflow with hyperparameters, evaluation metrics (PR-AUC, F1 score), and model artifacts. A FastAPI application loads the current production model at startup and exposes a `/predict` endpoint that returns fraud probability and SHAP explanations for a single transaction. A Streamlit dashboard provides a user interface for investigators to submit transactions and view explanations.

The pipeline is designed for reproducibility. All dependencies are pinned in `requirements.txt` with explicit versions (e.g., `lightgbm==4.6.0`). All cells are ordered to execute sequentially, producing the feature-engineered dataset, trained model, evaluation metrics, and SHAP plots.

V. ANALYSIS AND RESULTS

A. Headline results

Table I reports test-set performance on the held-out test set (20% of data, stratified to preserve the 0.13% fraud rate). Our LightGBM model with class weighting (`scale_pos_weight` = 773) achieves near-perfect fraud detection. The majority

TABLE I
TEST SET PERFORMANCE (20% HOLDOUT, STRATIFIED).

Metric	LightGBM	Majority Baseline
PR-AUC	0.9986	0.0013
F1 Score	0.9988	0.0000
Recall	0.9976	0.0000
Precision	1.0000	0.0000
Accuracy	1.0000	0.9987

TABLE II
CONFUSION MATRIX ON TEST SET.

Actual	Predicted	
	Legitimate	Fraud
Legitimate	1,270,881	0
Fraud	4	1,639

class baseline, predicting "legitimate" for every transaction, achieves 99.87% accuracy but detects zero fraud, demonstrating why accuracy is misleading for imbalanced problems.

The LightGBM model correctly identifies 99.76% of fraudulent transactions (recall = 0.9976) with no false positives (precision = 1.0000). The single misclassified fraud case (4 false negatives out of 1,643 fraud transactions) corresponds to a transaction with atypical balance patterns that did not trigger the balance engineering features.

B. Confusion Matrix

Figure II shows the confusion matrix for the test set predictions. The model achieved perfect precision (no false positives) and near-perfect recall (only 4 false negatives out of 1,643 fraud transactions).

C. Error decomposition

Although the model achieves near-perfect performance, examining the 4 false negatives (fraudulent transactions misclassified as legitimate) reveals useful insights.

All 4 false negatives were TRANSFER transactions where the sender's account balance did not go to zero after the transaction. In these cases, the fraudster transferred only a portion of the available balance rather than executing an exact drain. Consequently, key balance engineering features, particularly `orig_zeroed` and `exact_drain` did not trigger.

This suggests that while the PaySim dataset primarily models fraud as complete account draining, a small fraction of fraudulent transactions (0.24% of all fraud cases) follow a different pattern. In a real-world deployment, these cases would require additional features (e.g., velocity of transactions, connection to known fraudulent accounts) to be detected.

No false positives were generated, meaning the model never flagged a legitimate transaction as fraudulent. This could be seen as desirable because false positives waste investigator time and erode customer trust. However, the threshold could be lowered to increase recall (catching more fraud) at the cost of introducing some false positives.

D. Feature importance

LightGBM’s split-gain importance, corroborated by SHAP analysis, ranks balance engineering features as the top contributors to model performance. Figure ?? shows that `orig_balance_error` (expected minus actual sender balance), `exact_drain` (transaction amount equals sender’s entire balance), and `orig_zeroed` (sender balance equals zero after transaction) are the three most important features.

The dominance of balance engineering features aligns with our ablation study finding that removing this feature group causes PR-AUC to drop from 0.9986 to 0.5019 — essentially random performance. This confirms that the PaySim dataset models fraud primarily as account draining events.

Other feature groups contributed minimally:

- **Time features** (`hour_of_day`, `hour_of_week`, `is_night`) had negligible aggregate importance, suggesting fraud in PaySim is not temporally patterned.
- **Transaction type features** (`type_enc`, `is_risky_type`) showed low importance because all fraud occurs in TRANSFER and CASH_OUT; the binary `is_risky_type` flag already captures this signal completely.
- **Log transforms** (`log_amount`, `log_oldbal_orig`, etc.) provided modest normalization benefits but did not drive predictions independently.
- **Account type features** (`orig_is_customer`, `dest_is_customer`, `dest_is_merchant`) showed the lowest importance, indicating that customer vs. merchant status is not a strong fraud indicator in PaySim.

E. Limitations

Our evaluation, while rigorous within the PaySim environment, has several limitations that define the scope of our conclusions.

Synthetic data. PaySim is a simulated dataset, not real transaction data from a financial institution. While calibrated to replicate statistical properties of real mobile money transactions, fraud patterns in production systems are more complex, evolve over time, and may not exhibit the same account-draining signature that dominates PaySim. Our model’s performance on real-world data may be substantially lower, and validation on real transaction data would be necessary before deployment.

No temporal validation. PaySim lacks genuine timestamps; the `step` column represents hours since simulation start without calendar alignment. We used random stratified splitting rather than temporal (time-based) splitting. In a real-world deployment, models must be validated on future data to prevent look-ahead bias, and concept drift would require continuous monitoring and retraining.

Balance features dominate. Our ablation study revealed that balance engineering features account for nearly all predictive performance. Removing these features dropped PR-AUC from 0.9986 to 0.5019 — essentially random performance. This suggests the model may not generalize to fraud patterns

that do not involve account draining, such as coordinated fraud rings, account takeovers, or synthetic identity fraud. Additional feature engineering would be required to detect these patterns.

No network or behavioral features. We did not engineer features capturing transaction networks (e.g., connections between accounts, transaction velocity to common destinations) or long-term customer behavior patterns. Such features might detect fraud rings and account takeovers that our current model misses. Future work should incorporate graph-based features.

No cost-sensitive threshold optimization. While we tuned the decision threshold to maximize F1 score, real-world fraud detection involves explicit costs: false negatives (missed fraud) cause direct financial losses; false positives (legitimate transactions flagged) waste investigator time and may inconvenience customers. A cost-sensitive optimization would better align the model with business objectives.

Point predictions only. Our model produces fraud probabilities but not calibrated uncertainty estimates. A well-calibrated probabilistic forecast would allow investigators to prioritize high-uncertainty cases for manual review. This remains future work.

Single dataset. We trained and tested on PaySim only. Cross-validation on additional fraud detection datasets (e.g., the Financial Transactions Dataset from Kaggle, the Cifer Fraud Detection Dataset from Hugging Face) would strengthen claims of generalizability.

VI. CONCLUSIONS AND INSIGHTS

A pragmatic combination of Polars-based data processing, balance-focused feature engineering, and gradient-boosted classification delivered a fraud detection on the PaySim synthetic dataset. The three key takeaways:

- 1) **Handle class imbalance aggressively.** With only 0.13% fraud, a naive classifier achieves 99.87% accuracy while catching zero fraud. Using `scale_pos_weight = 773` (the imbalance ratio) in LightGBM, tuning the decision threshold on the precision-recall curve, and evaluating with PR-AUC rather than accuracy proved essential. Teams working on similar imbalanced problems should prioritize cost-sensitive learning over sampling methods.
- 2) **Balance engineering does the heavy lifting.** Our ablation study revealed that removing balance features caused PR-AUC to drop from 0.9986 to 0.5019 — essentially random performance. Features detecting account draining (`orig_zeroed`, `exact_drain`) captured the signature fraud pattern: 98% of fraudulent transactions leave the sender’s balance at exactly zero. Domain-specific features matter more than generic features like time or transaction type.
- 3) **Engineering around the model matters.** Packaging the pipeline with MLflow tracking, SHAP explainability, pinned dependencies, and a FastAPI service makes the work reproducible, explainable, and deployable. This is the difference between a course artifact and something that could plausibly run in a real operations setting.

VII. REFLECTION ON PROJECT PROCESS AND CHALLENGES

The group divided work along the pipeline. Three challenges are worth naming:

- **Handling extreme class imbalance.** Our initial models predicted "legitimate" for every transaction, achieving 99.87% accuracy while detecting zero fraud. Standard machine learning algorithms optimized for accuracy fail catastrophically on data with 0.13% fraud rate. We resolved this by implementing cost-sensitive learning via LightGBM's `scale_pos_weight` parameter set to the imbalance ratio (773:1), tuning the decision threshold on the precision-recall curve, and using PR-AUC instead of accuracy for evaluation.
- **Feature engineering for account draining detection.** Our first model used only raw transaction features (amount, step, transaction type) and performed poorly (PR-AUC below 0.6). Manual analysis of fraudulent transactions revealed that 98% leave the sender's balance at exactly zero. We engineered nine balance-based features capturing this pattern, including `orig_zeroed`, `exact_drain`, and `orig_balance_error`. This improved PR-AUC from 0.6 to 0.9986 — a 66% relative improvement.
- **Data leakage from `isFlaggedFraud`.** Early exploratory analysis showed perfect correlation between `isFlaggedFraud` and certain fraud cases. Including this column would have created a model that simply memorized the rule, achieving perfect training accuracy but failing completely on any fraud pattern not captured by that static threshold. We recognized this as data leakage and excluded the column entirely from the feature set.

Contributions were tracked in a shared board and validated in the individual interviews. We used AI tools (Claude, Copilot, and Deepseek) for drafting documentation, generating unit-test skeletons, and clarifying error messages. All modeling decisions, feature choices, and final analysis were made by the authors.

VIII. QUALITY OF CODE AND REPRODUCIBILITY

The source repository is organized as follows:

- `Project_4_Enterprise_Fraud_Detection_Platform.ipynb`: Main Jupyter notebook containing the complete pipeline: data ingestion, feature engineering, model training, evaluation, and SHAP analysis.
- `app.py`: FastAPI application for serving the trained model, exposing a `/predict` endpoint that returns fraud probability and SHAP explanations.
- `fraud_dashboard.py`: Streamlit dashboard providing a user interface for investigators to submit transactions and view predictions with explanations.
- `mlflow_logging_code.py`: MLflow logging utilities for tracking hyperparameters, metrics, and model artifacts.

- `test_api.py`: Pytest suite for testing the FastAPI endpoint, including smoke tests for valid and invalid inputs.
- `requirements.txt`: Pinned dependencies with explicit versions.
- `.gitignore`: Configured to exclude data files, virtual environments, cache directories, and MLflow artifacts.

The pipeline is reproducible from a clean clone using the following commands:

```
git clone <repository-url>
cd <repository-directory>
python -m venv venv
source venv/bin/activate
# or
venv\Scripts\activate on Windows
pip install -r requirements.txt
jupyter notebook Project_4_Enterprise_Fraud_Detection_Platform.ipynb
```

Executing all cells in the notebook sequentially reproduces the feature-engineered dataset, trained model, evaluation metrics (PR-AUC = 0.9986, F1 = 0.9988), confusion matrix, ablation study results, and SHAP plots reported in this paper.

For API deployment, the FastAPI server can be started with:

```
uvicorn app:app --reload
```

The Streamlit dashboard can be launched with:

```
streamlit run fraud_dashboard.py
```

The repository is hosted on GitHub and a tagged release ("final-submission") has been created. The repository URL and release are supplied with this report. No large data files are committed to the repository; the PaySim dataset is downloaded automatically via KaggleHub when the notebook is run.

ACKNOWLEDGEMENTS

We thank Mr. Sergio Mathurin and Mr. Anton Greenridge for guidance throughout the course, and Edgar Lopez-Rojas for creating and making publicly available the sample PaySim dataset.

REFERENCES

- [1] E. Lopez-Rojas and A. Elmir, "PaySim: Synthetic Financial Dataset for Fraud Detection", 2017. [Online]. Available: <https://www.kaggle.com/datasets/ealaxi/paysim1>
- [2] M. Ahmed, A. N. Mahmood, and M. R. Islam, "A survey of anomaly detection techniques in financial fraud detection," 2016.
- [3] R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Statistical Science*, 2002.
- [4] F. T. Liu, K. M. Ting, and Z. H. Zhou, "Isolation-based anomaly detection," *ACM TKDD*, 2012.
- [5] S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. C. Westland, "Data mining for credit card fraud: A comparative study," *Decision Support Systems*, 2011.
- [6] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *JAIR*, 2002.
- [7] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *NeurIPS*, 2017.
- [8] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *KDD*, 2016.

- [9] J. Davis and M. Goadrich, "The relationship between Precision-Recall and ROC curves," ICML, 2006.
- [10] S. M. Lundberg and S. I. Lee, "A unified approach to interpreting model predictions," NeurIPS, 2017.

APPENDIX

AI TOOLS USED

We used Claude, GitHub Copilot and Deepseek for (i) drafting skeletons of unit tests and docstrings, (ii) debugging error messages, and (iii) copy-editing sections of this report. All modeling choices, feature selection, evaluation, and analytical claims were made and verified by the authors.